Introduction to OpenLane flow

INTRODUCTION:

OpenLane is a free tool that turns Verilog code into a chip layout ready for making real ICs. You give it your design files and simple settings like clock speed, then run one command to automatically do synthesis, placement, wiring, checks, and output the final manufacturing files. It automates the complete RTL-to-GDSII digital ASIC design flow in a unified, scriptable environment. Designers provide Verilog source files and a configuration specifying parameters like clock period, core utilization, and process design kit, then execute a single command to generate manufacturing-ready GDSII along with comprehensive signoff reports covering timing, power, DRC, and LVS.

APPLICATION:

OpenLane finds primary application in automating digital ASIC design, enabling students, researchers, and small teams to create manufacturable chip layouts from RTL code without expensive commercial licenses. It supports educational projects, RISC-V processor implementations, test chip tape-outs like SoC integration flows for open hardware initiatives. Additional uses include hardening macros, tiny tapeout shuttles, and prototyping custom IP blocks on accessible PDKs for low-volume fabrication.

OBJECTIVE:

After completing this lab, you will be able to:

- Understand a flow from RTL-to-GDSII
- Navigate OpenLane directory structure and multi-run iteration process.
- Customize flow variables for design optimization and convergence
- Debug common issues like congestion, timing violations and DRC errors
- Export final layout views for verification and tapeout preparation.

PROCEDURE TO CREATE A LAB:

Step 1: Set up OpenLane flow

1.1: Invoke OpenLane in the terminal using the command: `openlane`

1.2: Open folder- designs: `cd designs`

Step 2: Set up Project Directory

2.1: Create a directory for your lab: `mkdir <project_name>`

2.2: Open lab directory: `cd <project_name>`

2.3: Create directory for project: `mkdir decoder`

2.4: Open project directory: `cd decoder`

2.5: Inside project directory, create folder: `mkdir src`

2.6: Open folder: `cd src`

Step 3: Create Design file

3.1: Open vi editor for design with command: `vi decoder.v`

RTL Design Code:

```
module decoder (
    input wire [3:0] in,
    output reg [15:0] out
);
    always @(*) begin
        out = 16'b0;  // Default: all outputs low
        case (in)
            4'b0000: out[ 0] = 1'b1;
            4'b0001: out[ 1] = 1'b1;
            4'b0010: out[ 2] = 1'b1;
```

```
            4'b0011: out[ 3] = 1'b1;
            4'b0100: out[ 4] = 1'b1;
            4'b0101: out[ 5] = 1'b1;
            4'b0110: out[ 6] = 1'b1;
            4'b0111: out[ 7] = 1'b1;
            4'b1000: out[ 8] = 1'b1;
            4'b1001: out[ 9] = 1'b1;
            4'b1010: out[10] = 1'b1;
            4'b1011: out[11] = 1'b1;
            4'b1100: out[12] = 1'b1;
            4'b1101: out[13] = 1'b1;
            4'b1110: out[14] = 1'b1;
            4'b1111: out[15] = 1'b1;
            default: out = 16'b0;
        endcase
    end
    endmodule
```

3.2: save the file using command- `:wq`

Step 4: Create .json file

4.1: Open vi editor: `vi config.json`

```
{
    "DESIGN_NAME": "decoder",
    "VERILOG_FILES": "dir::src/*.v",
    "RUN_CTS": false,
    "CLOCK_PORT": null,
    "PL_RANDOM_GLB_PLACEMENT": true,
    "FP_SIZING": "absolute",
    "DIE_AREA": "0 0 34.5 57.12",
    "PL_TARGET_DENSITY": 0.75,
    "FP_PDN_AUTO_ADJUST": false,
    "FP_PDN_VPITCH": 25,
```

```
    "FP_PDN_HPITCH": 25,
    "FP_PDN_VOFFSET": 5,
    "FP_PDN_HOFFSET": 5,
    "DIODE_INSERTION_STRATEGY": 3
}
```

5. Use the following commands to run the flow:

#To run OpenLane in one go

```
flow.tcl -design <design_name>
```

OpenLane to start the design flow in interactive mode

```
i)    flow.tcl -interactive
```

Used in the OpenLane interactive shell to prepare your design for the digital IC design flow.

```
ii)   prep -design decoder
```

Runs yosys synthesis on the current design as well as OpenSTA timing analysis on the generated netlist

```
iii)  run_synthesis
```

To initiate floorplanning

```
iv)   run_floorplan
```

To initiate placement/Runs global placement

```
v)    run_placement
```

To initiate Clock Tree Synthesis (CTS) using openroad on the processed design.

```
vi)   run_cts
```

To run the routing process

```
vii)  run_routing
```

To streams the final GDS and layout view

```
viii)   run_magic
```

To runs spice extractions on the processed design.

```
ix)   run_magic_spice_export
```

To runs a drc check

```
x)    run_magic_drc
```

Streams the back-up final GDS-II, generates a PNG screenshot, then runs Klayout DRC deck on it.

```
xi)  run_klayout
```

Runs an lvs check between an extracted spice netlist and the current verilog netlist of the processed design

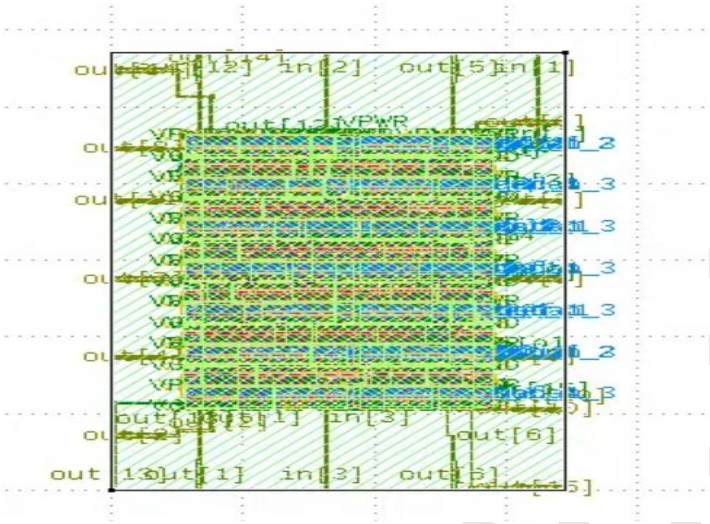
```
xii) run_lvs
```

OBSERVATION:

- **Synthesis results:** Gate count, cell area utilization, and logic optimization statistics from Yosys netlist generation.
- **Floorplan metrics:** Die/core dimensions, utilization percentage, IO pin placement, and macro positioning.
- **Placement density:** Cell density maps, congestion hotspots, and placement runtime.
- **Clock tree quality:** CTS skew, insertion delay, buffer/inverter count, and clock network balance.
- **Routing completion**: Global/detailed routing overflow, wirelength estimates, and via counts.
- **Timing slacks:** Setup/hold margins across corners, critical path locations, and WNS/TNS values.
- **DRC/LVS violations:** Layer violations, short/open counts, and layout vs. schematic mismatches.
- **Power estimates:** Dynamic leakage breakdown by cell type and total power consumption.
- **Final GDSII:** Stream file size, layer hierarchy, and tapeout readiness confirmation.

OUTPUT:

```
[INFO]: Running Circuit Validity Checker ERC (log: runs/RUN_2026.01.20_08.56.49/logs/signoff/39-erc_screen.log)...
[INFO]: Saving current set of views in 'runs/RUN_2026.01.20_08.56.49/results/final'...
[INFO]: Saving runtime environment...
[INFO]: Generating final set of reports...
[INFO]: Created manufacturability report at 'runs/RUN_2026.01.20_08.56.49/reports/manufacturability.rpt'.
[INFO]: Created metrics report at 'runs/RUN_2026.01.20_08.56.49/reports/metrics.csv'.
[INFO]: There are no max slew, max fanout or max capacitance violations in the design at the typical corner.
[INFO]: There are no hold violations in the design at the typical corner.
[INFO]: There are no setup violations in the design at the typical corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
```



After the flow from RTL-to-GDSII, we will be able to see the layout of our design.

CONCLUSION:

OpenLane successfully implements the RTL design through complete RTL-to-GDSII flow including synthesis, floorplanning, placement, CTS, routing, and signoff verification. The analysis confirms DRC clean layout, LVS matched schematic, and positive timing slacks across all corners with no violations detected. The design achieves timing closure at the target clock frequency while meeting power and area constraints. Overall, OpenLane validates production-ready GDSII output suitable for ASIC tapeout and manufacturing.